Лабораторная работа 1. Задание 2: задача N тел

1. Постановка задачи

Написать параллельную программу, которая считает траектории N материальных точек от 0 до t_end. На точку заданы масса, начальные координаты и скорость. Сила берётся из закона тяготения, шаг по времени — метод Эйлера первого порядка.

Запуск: ./nbody t_end input.txt [nthreads] [dt] [output.csv]. nthreads — число рабочих потоков. dt — шаг по времени в секундах: на столько сдвигается момент времени за один шаг Эйлера. output.csv — файл с траекторией: в каждой строке момент времени и координаты x, y всех тел. Аргументы в квадратных скобках можно не передавать: тогда потоков 4, шаг 0.01 с, файл trajectory.csv в текущей папке. Папки inputs/ в репозитории нет. make run создаёт inputs/ и results/, собирает inputs/test_100.txt на 100 тел, если этого файла ещё нет, и запускает расчёт.

Пример: ./gen_input 100 demo.txt, затем ./nbody 1.0 demo.txt 4 0.01 out.csv. Это 100 тел, считать до t_end = 1 с, 4 потока, шаг 0.01 с, траектория в out.csv. Оба файла появляются в текущем каталоге.

Во входном файле первая строка — n, дальше n строк по семь чисел: масса, x y z, vx vy vz. В методичке формулы Эйлера записаны для x и y, во входном файле есть ещё z, поэтому движение считается в трёхмерном пространстве. В CSV пишутся только x и y, как просит вывод:

```
t, x1, y1, x2, y2, ..., xn, yn
```

Каждая строка — координаты в один момент времени. Шаг dt в условии не задан; в замерах он равен 0.01 с.

Входной файл собирает gen_input.c. Случайные числа даёт my_drand из my_rand.c в каталоге задания, начальное состояние seed = 42. Масса от 1e20 до 1e24, координаты от -1e11 до 1e11, скорости от -1e3 до 1e3. Запуск: ./gen_input n имя_файла, где n — сколько тел записать в файл. Пример: ./gen_input 100 demo.txt.

2. Сведения о реализованном алгоритме

Сила на тело i со стороны тела j:

```
F = G * mi * mj * (rj - ri) / |rj - ri|^3
G = 6.67430e-11
```

Пара (i, j) считается один раз. На i прибавляется F, на j прибавляется -F. Это третий закон Ньютона, он сокращает число пар вдвое. К квадрату расстояния добавляется 1e-30, чтобы не делить на ноль, если две точки совпали. На обычных расстояниях это слагаемое ничего не меняет.

Координаты и скорости на шаге n берутся из значений шага n−1. Сначала все силы считаются по старым координатам, и только потом обновляются положения и скорости:

```
x  = x  + vx * dt
vx = vx + Fx / m * dt
```

То же для y и z. Если сдвинуть тело раньше, чем посчитаны все силы, разные пары на одном шаге увидят разные положения.

Распараллеливание:

- Пар n(n−1)/2. Этот диапазон делится между потоками поровну.

- Поток не пишет в общий массив сил: у каждого свой `Fx_loc[tid]`. Иначе два потока, считающие разные пары одного тела, затирали бы силу друг другу.
- На каждом шаге четыре барьера: закончили пары; сложили силы по своим телам; обновили координаты и скорости; главный поток записал строку CSV и при необходимости поставил флаг остановки.
- Главный поток пары не считает. Он нужен, чтобы шаг не разъехался: силу нельзя суммировать, пока кто-то ещё пишет локальный массив, и нельзя читать координаты для файла, пока кто-то их меняет.

3. Пояснения к коду

Программа — файл `nbody.c`. Сборка: `gcc -O2 -pthread -lm nbody.c -o nbody`. Генератор входного файла — `gen_input.c` и `my_rand.c` в этом же каталоге.

Подготовка. `read_input` читает `n` и семь чисел на тело. Память под массы, координаты, скорости и силы выделяется до запуска потоков. `alloc_local` даёт каждому потоку свои массивы сил.

Функция потока `worker`. Номер потока приходит в структуре вместе с указателем на общие данные. Поток берёт свой отрезок пар `[p_start, p_end)` и свою группу тел `[i_start, i_end)`. В цикле шага он:

1. обнуляет свой локальный массив сил и считает пары из своего отрезка, прибавляя `F` к телу `i` и `-F` к телу `j`;
2. после барьера складывает куски силы всех потоков для своих тел и записывает полную силу в общий массив;
3. после второго барьера обновляет координаты и скорости только своих тел;
4. проходит ещё два барьера: один после обновления, второй — пока главный поток пишет CSV. Если флаг остановки уже стоит, поток выходит.

Функция `main`. Читает аргументы, создаёт потоки и барьер на `nthreads + 1` участников (рабочие потоки и главный). Время берётся через `clock_gettime(CLOCK_MONOTONIC)` вокруг цикла по шагам. На каждом шаге главный поток трижды ждёт барьер, увеличивает время на `dt`, пишет строку CSV и на четвёртом барьере отпускает рабочие потоки. Когда `t` достигло `t_end`, перед этим барьером ставится флаг остановки.

Вывод программы: на экран — время цикла по шагам в секундах; в CSV — строка на каждый момент, включая `t = 0`. Чтение входного файла в замер не входит. Запись CSV входит: строка пишется на каждом шаге внутри цикла.

4. Условия измерений

- Процессор: Intel Core i5-13420H. В WSL он виден как 6 ядер и 12 потоков.
- Система: Ubuntu 22.04.5, ядро 6.6.87.1-microsoft-standard-WSL2, памяти выделено 3.7 ГБ. Сборка: gcc 11.4.0, флаги `-O2 -pthread -lm`.
- Время — `clock_gettime(CLOCK_MONOTONIC)` вокруг цикла по шагам. Каждая пара «число частиц, число потоков» запускалась 5 раз, в таблицу положен минимум.
- Шаг везде `dt = 0.01` с. Три размера задачи: 1000 частиц и `t_end = 1.0` (100 шагов), 5000 частиц и `t_end = 1.0`, 20 000 частиц и `t_end = 0.2` (20 шагов).

5. Результаты замеров

Ускорение $S = T_1 / T_p$, эффективность $E = S / p$. T_1 — минимум на одном потоке, T_p — минимум на p потоках.

1000 частиц, 100 шагов:

Потоки p	Время T (минимум), с	Ускорение S	Эффективность E
1	0.8923	1.00	1.00
2	0.8552	1.04	0.52
4	0.6161	1.45	0.36
6	0.5987	1.49	0.25
8	0.5722	1.56	0.19
12	0.5575	1.60	0.13

5000 частиц, `t_end = 1.0`:

Потоки p	Время T (минимум), с	Ускорение S	Эффективность E
1	17.3158	1.00	1.00
2	10.6001	1.63	0.82
4	7.5942	2.28	0.57
6	6.1042	2.84	0.47
8	4.9023	3.53	0.44
12	4.6741	3.70	0.31

20 000 частиц, 20 шагов:

Потоки p	Время T (минимум), с	Ускорение S	Эффективность E
1	63.9812	1.00	1.00
2	41.5130	1.54	0.77
4	23.9883	2.67	0.67
6	19.0094	3.37	0.56
8	15.1357	4.23	0.53
12	14.2081	4.50	0.38

6. Выводы

Программа считает траектории по закону тяготения и методу Эйлера и записывает их в CSV. Пара тел обрабатывается один раз, с учётом третьего закона Ньютона: без этого пар было бы вдвое больше.

На 20 000 частицах пар около 200 миллионов на шаг, и 12 потоков дают ускорение 4.50 (эффективность 0.38). На 5000 частицах ускорение 3.70. На 1000 частицах только 1.60: перебор пар короткий, а на каждом шаге четыре барьера и запись CSV уже сравнимы с самим счётом. После 4 потоков время там почти не падает.

В WSL процессор виден как 6 ядер по 2 потока. На больших задачах время заметно уменьшается примерно до 6 потоков; на 8 и 12 второе логическое ядро делит те же блоки исполнения, поэтому прирост меньше. Эффективность снижается во всех трёх сериях: барьеры и главный поток, который на шаге пишет файл, от числа потоков быстрее не становятся.